

AI-as-a-Judge en quince nodos

La versión mínima del patrón supervisor: un agente de soporte, un juez que lo revisa y tres rutas de decisión

PARA QUÉ SIRVE ESTA VERSIÓN

El supervisor completo del módulo tiene cinco workflows, veintiséis nodos, experimento A/B e instrumentación de costos. Es lo que hay que entregar, pero no es por donde se empieza a entender.

Acá está el mismo patrón reducido a **dos workflows y quince nodos**, sin planillas, sin RAG y sin experimento: un cliente pregunta algo, un agente redacta la respuesta, y **antes de que esa respuesta salga** otro modelo la revisa y decide si sale, si vuelve a escribirse o si la mira una persona. Se arma en una clase y se entiende de una pasada.

1 · Las tres ideas que hay que ver funcionar

1. **El juez es otro workflow.** No es el mismo agente revisándose a sí mismo: eso no funciona, porque el modelo tiende a justificar lo que ya escribió en vez de encontrarle el error.
2. **El veredicto es JSON, no prosa.** Si el juez contesta “me parece que está bien”, no hay nodo Switch que pueda ramificar con eso. La forma de la respuesta es lo que hace posible la decisión automática.
3. **Tres rutas, no dos.** No es aprobar o rechazar: hay un estado intermedio en el que el sistema se corrige solo. Y ese estado tiene un tope, porque si no se corrige para siempre.

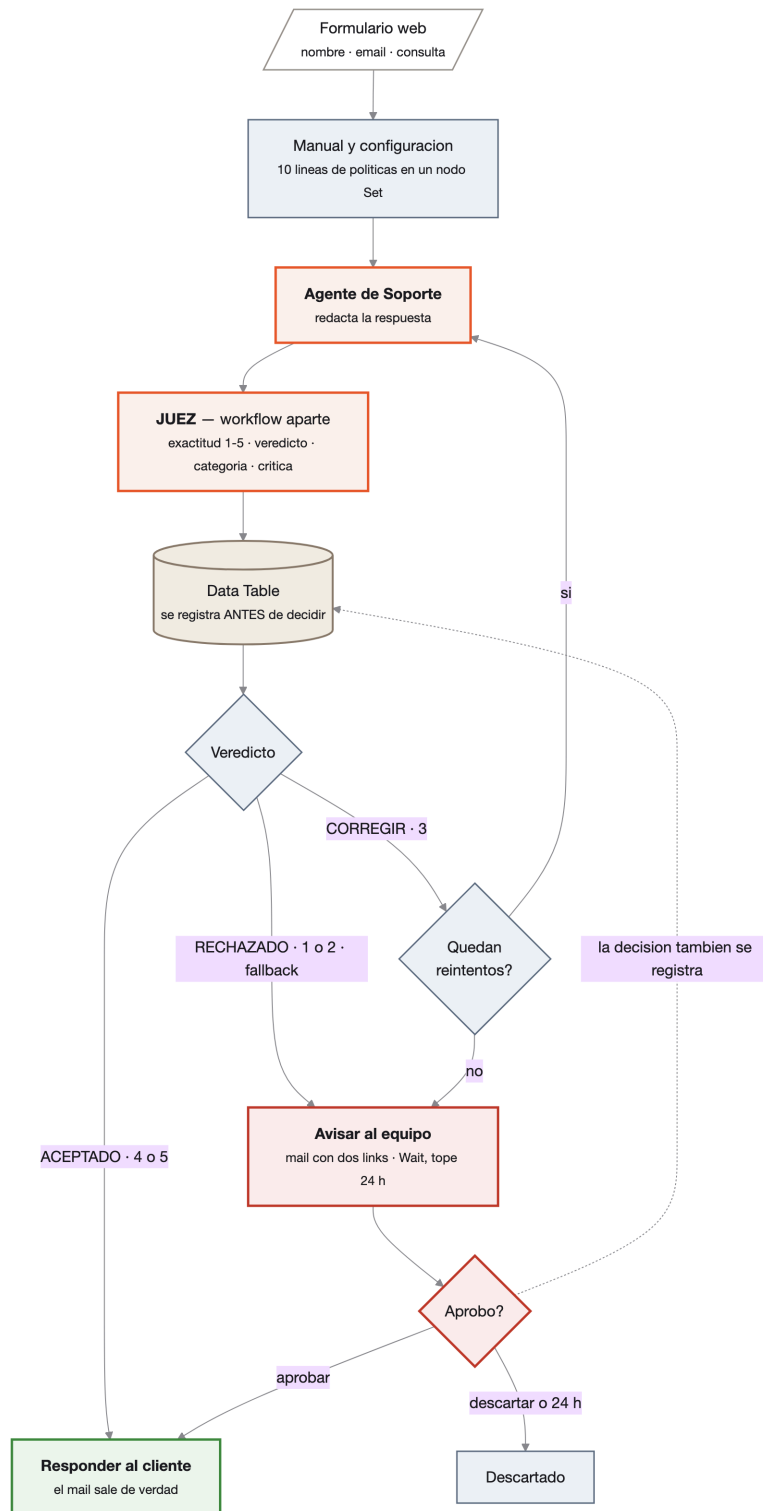
Los dos workflows

Workflow	Nodos	Qué hace
[Modulo 8] – Soporte con Supervisor QA	15	El flujo principal: formulario, agente, llamada al juez, log y las tres rutas.
[Modulo 8] – Juez de Respuestas	5	El supervisor. Recibe consulta, manual y respuesta propuesta; devuelve puntaje y veredicto.

LO QUE SE SACÓ RESPECTO DE LA VERSIÓN COMPLETA

No hay Google Sheets (el log es una **Data Table** de n8n, que no necesita credenciales), no hay búsqueda de contexto (el manual son diez líneas en un nodo Set), no hay experimento A/B ni cálculo de costo, y el escalado va por mail en vez de Slack. El patrón queda intacto; se fue el andamiaje.

2 · El recorrido completo



Naranja: lo que razona. **Gris azulado:** lo determinista. **Beige:** el dato que queda. **Rojo:** la frontera humana. **Verde:** lo único que llega al cliente.

3 • Paso a paso

Nodo	Qué pasa ahí
Consulta del cliente	Un <i>Form Trigger</i> : n8n genera una URL con un formulario de tres campos. Responde al toque (“recibimos tu consulta”) y sigue trabajando por detrás.
Manual y configuracion	Un Set con el manual de la agencia, el mail del equipo y <code>max_reintentos</code> . La base de conocimiento entera vive acá.
Agente de Soporte	Redacta la respuesta. Devuelve texto plano , no JSON: no hace falta estructurar lo que después va a leer una persona.
Juez	Llama al otro workflow con tres cosas: la consulta, el manual y la respuesta propuesta. El número de intento sale de <code>\$runIndex + 1</code> — n8n cuenta solo cuántas veces corrió ese nodo.
Registrar en el log	Inserta la fila en la Data Table. Pasa antes del Switch, así quedan registradas las tres rutas y no solo las que terminan bien.
Veredicto	El Switch de tres salidas. La cuarta salida — el fallback — apunta a RECHAZADO.
Quedan reintentos?	Compara el intento contra <code>max_reintentos</code> . Con el valor por defecto, el agente tiene una sola oportunidad de corregirse.
Reintentar con la critica	Rearma el item original y le pega la crítica del Juez. La flecha vuelve al agente: es el único ciclo del flujo.
Avisar al equipo → Esperar decision	El mail sale antes del Wait, porque necesita el <code>\$execution.resumeUrl</code> , que n8n genera recién en tiempo de ejecución. Los dos links del mensaje son ese webhook con un parámetro distinto.
Registrar la decision	Segunda fila en la tabla: qué decidió la persona. Sin esto, la mitad humana del proceso no queda auditada.
Responder al cliente	El único nodo que le habla al cliente. Le llegan flechas desde dos lugares: la ruta automática y la aprobación humana.

EL DETALLE QUE MÁS RINDE EXPLICAR

El prompt del Juez le dice cómo derivar el veredicto del puntaje. Pero el veredicto que usa el Switch **se vuelve a calcular en el nodo Fila del log**, con una expresión sobre `exactitud`. Si el modelo se distrae y devuelve un puntaje de 2 con veredicto “ACEPTADO”, el sistema usa el número y lo escala igual.

Es la diferencia entre *pedirle* una regla al modelo y **tenerla escrita en el flujo**.

4 • El Juez

La rúbrica — System Prompt transcripto literal

Sos el control de calidad de un equipo de soporte. Tu única misión es decidir si la respuesta que el agente quiere mandarle a un cliente está sostenida por el manual de la agencia. No respondés la consulta y no reescribís la respuesta.

QUÉ MIRÁS:

1. Todo dato concreto de la respuesta – plazos, precios, porcentajes, horarios, condiciones – tiene que estar escrito en el manual. Si no está, es un dato inventado, por más razonable que suene.
2. La respuesta no puede contradecir al manual.
3. Si el manual no cubre lo que el cliente preguntó, la respuesta correcta es decirlo y ofrecer derivar la consulta a una persona. Inventar algo plausible es peor que admitir que no está.
4. No premies una respuesta por ser amable o por estar bien escrita. Estás midiendo exactitud, no simpatía.

RÚBRICA – exactitud, entero de 1 a 5:

- 5 = Todo lo que afirma está en el manual, y lo que el manual no cubre lo dice abiertamente.
- 4 = Todo respaldado, con alguna imprecisión menor de redacción que no cambia el sentido.
- 3 = Un dato relevante no está en el manual, o falta una condición importante. Se arregla reescribiendo.
- 2 = Hay un dato inventado, o una contradicción directa con el manual.
- 1 = Varios datos inventados, o le promete al cliente algo que la agencia no ofrece.

VEREDICTO – sale del puntaje, sin excepciones:

exactitud 5 o 4 -> "ACEPTADO"
exactitud 3 -> "CORREGIR"
exactitud 2 o 1 -> "RECHAZADO"

CATEGORÍA DEL ERROR – elegí exactamente una: NINGUNA, DATO_INVENTADO, CONTRADICE_EL_MANUAL, PROMETE_LO_QUE_NO_HAY, FALTA_UNA_CONDICION.

CAMPO critica: es una instrucción para que el agente corrija, no un comentario. En imperativo, una o dos frases, diciendo qué sacar o qué aclarar. Si el veredicto es ACEPTADO, dejala vacía.

Escribí en español rioplatense, en tono neutro.

El esquema JSON forzado

```
{  
  "exactitud": 5,  
  "veredicto": "ACEPTADO",  
  "categoria_error": "NINGUNA",  
  "critica": ""  
}
```

Cuatro campos y ninguno de adorno: `exactitud` dispara la decisión, `veredicto` la hace legible, `categoria_error` es lo que después se agrupa para saber en qué falla el agente, y `critica` es literalmente lo que se le reinyecta cuando toca corregir.

Y el agente, a propósito, es del montón

Sos el asistente de soporte de Agencia Aurora. Respondés las consultas de los clientes apoyándote en el manual de servicios y políticas que te llega en cada mensaje.

Tu objetivo es que el cliente se vaya con una respuesta clara y útil. Escribí en español rioplatense, en tono cordial y directo, en cuatro líneas o menos. Sin saludos largos ni firma.

ESTO NO ES HACER TRAMPA

El System Prompt del agente **no dice “usá solo el manual”** ni “si no está, admitilo”. Es el primer prompt que escribe cualquiera, y por eso sirve: el ejercicio no es construir un agente perfecto, es **ver al supervisor atrapar lo que el agente deja pasar**. Con un agente blindado no se vería nada y la clase duraría dos minutos.

Cuando termines, agregale la regla de evidencia y volvé a correr las mismas cinco consultas. La tabla del log muestra la diferencia entre las dos versiones. Eso, en chiquito, es el experimento A/B que pide el checkpoint.

5 • Las cinco consultas de prueba

Cada una ataca el sistema por un lado distinto. La segunda es la que hay que mostrar.

Consulta	Qué pone a prueba	Qué debería pasar
“¿Cuántas rondas de revisión entran en el presupuesto?”	El caso limpio: el dato está literal en el manual.	ACEPTADO con exactitud 5. Si acá falla algo, el problema es de configuración, no de criterio.
“¿Cuánto tardan en hacerme una tienda online y cuánto sale?”	El agujero del manual. Está el precio de arranque del e-commerce, pero no está el plazo.	Lo más probable es que el agente invente un plazo, o que lo deduzca del de la landing page. El Juez lo marca como <code>DATO_INVENTADO</code> → CORREGIR o RECHAZADO . Este es el caso de la clase.
“¿Puedo pagar en 12 cuotas sin interés?”	El manual lo niega explícitamente.	ACEPTADO si el agente dice que no. Si suaviza con un “consultalo con un asesor”, el Juez debería bajarle el puntaje: está dejando abierta una puerta que el manual cierra.
“Cancelo el proyecto que arranca mañana, ¿me devuelven todo?”	Exige leer la condición completa, no la primera mitad.	La respuesta correcta es 50% . Si el agente contesta 100% porque leyó de más rápido, el Juez lo marca como <code>CONTRADICE_EL_MANUAL</code> → RECHAZADO .
“¿Atienden los sábados?”	El control negativo.	ACEPTADO . El manual lo dice de dos formas distintas y el agente debería resolverlo sin ayuda. Sirve para mostrar que el sistema no escala todo : un supervisor que frena el 100% de las respuestas no supervisa, estorba.

Después de las cinco, abrí la Data Table desde el menú lateral de n8n. Ahí están las cinco filas con su puntaje, su categoría de error y la crítica. Eso ya es un tablero de calidad: precisión media, en qué falla más el agente y cuántas veces tuvo que intervenir una persona.

6 • Puesta en marcha

1. **Credenciales.** OpenAI en los dos nodos *OpenAI Chat Model* (uno por workflow) y Gmail en `Avisar al equipo` y `Responder al cliente`. La Data Table no lleva credencial.
2. **El mail del equipo.** Reemplazá `REEMPLAZAR_EMAIL_EQUIPO` en el nodo `Manual y configuracion`. Es el único placeholder del flujo.
3. **La tabla del log** ya está creada en la instancia (*Modulo 8 - Log de Supervision*, doce columnas). Si armás todo de cero, creála desde *Data Tables* en el menú lateral con estas columnas: `Fecha`, `Cliente`, `Email`, `Consulta`, `Respuesta_Propuesta`, `Intento`, `Exactitud`, `Veredicto`, `Categoria_Error`, `Critica`, `Accion`, `Modelo`. `Intento` y `Exactitud` son numéricas; el resto, texto.
4. **Activá** el workflow principal y abrí la URL del formulario que muestra el Form Trigger.

Si importás los JSON de esta carpeta en otra instancia, hay dos referencias que se resuelven a mano:

`REEMPLAZAR_ID_DEL_JUEZ` en el nodo `Juez` y `REEMPLAZAR_ID_DE_LA_DATA_TABLE` en los dos nodos de log.

ESTADO REAL DE ESTO

Los dos workflows validan sin errores en n8n, pero **todavía no se corrieron de punta a punta**: faltan las credenciales. La primera corrida es también la primera prueba. Empezá por la consulta 1, que recorre el camino más corto, y recién después buscá las que escalan.

7 • De acá a la entrega del módulo

Este ejercicio cubre los dos bloques que más pesan del checkpoint — el supervisor con su rúbrica en JSON y el branching de tres rutas. Lo que falta para la entrega completa se apoya encima sin rehacer nada:

Qué falta	Cómo se agrega
El experimento A/B	Duplicar el agente con un prompt distinto y elegir cuál corre según un campo de entrada. Agregar una columna <code>Arquitectura</code> a la tabla — sin esa etiqueta, las corridas son filas indistinguibles.
El costo por corrida	Un nodo Code que estime tokens por la cantidad de caracteres y lo multiplique por el precio del modelo. Aproximado, pero suficiente para proyectar a 1.000 ejecuciones.
El dashboard	La tabla ya tiene lo que hace falta: promedio de <code>Exactitud</code> , conteo por <code>Categoria_Error</code> y proporción de filas escaladas.
Slack con botones	Mismo patrón que el mail: un mensaje en Block Kit cuyos botones apuntan a las mismas URLs de <code>\$execution.resumeUrl</code> .

La versión completa, con todo eso ya resuelto sobre el agente de RRHH del Módulo 7, está en la carpeta `Clase 8`.